

LINGUAGEM DE PROGRAMAÇÃO CLIENTE/SERVIDOR

Aula 02: Arquitetura cliente/servidor
Prof.: Me Francisco Erberto



Agenda

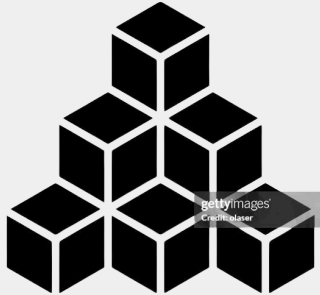
UNIDADE 1: Fundamentos da Arquitetura Cliente/Servidor

Aula	Conteúdo Teórico	Conteúdo Prático	Objetivo
2	Arquitetura cliente/servidor, HTTP, Requisição e resposta.	Simulação de requisições via navegador e DevTools.	Compreender o modelo cliente/servidor.
3	Protocolo HTTP: métodos, status code, headers, JSON.	Testes com Postman e APIs públicas.	Entender comunicação web.
4	Introdução ao Spring Boot. Conceito de servidor embarcado.	Criação do primeiro projeto via Spring Initializr. Hello World com @RestController.	Subir primeiro servidor.
5	Mapeamento de rotas: @GetMapping, @PostMapping.	Criar endpoints GET e POST.	Entender roteamento backend.
6	Estrutura de projeto Spring. Camadas Controller, Service e Model.	Organização de projeto em camadas.	Introduzir boas práticas arquiteturais.
7	JSON e serialização automática.	Criar API que retorna objetos Java como JSON.	Consolidar API REST básica.

Objetivo

- Compreender o modelo cliente/servidor
- Entender o papel do HTTP
- Identificar cliente, servidor e dados em sistemas reais
- Visualizar o fluxo completo de uma requisição

Introdução

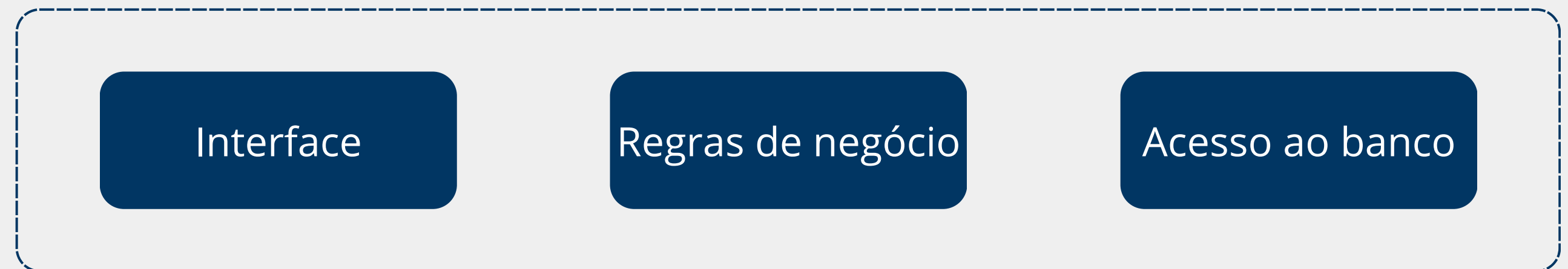
Anos 90: Desktop	Cliente/Servidor Clássico	SOA e SOAP	REST e APIs Web	Microserviços
				
Modelo 2 Camadas	Modelo 3 Camadas	Padronização Complexa	Modelo Atual	Cloud-Ready
Instalação local Forte acoplamento Cliente → Banco	Regras centralizadas no servidor Separação de responsabilidades Cli → Serv → Banco	Comunicação via XML Alto formalismo Excesso de complexidade	Protocolo HTTP e JSON Comunicação simplificada Separação clara Cli/Serv	Serviços pequenos e independentes Escalabilidade individual Comunicação via REST

■ ————— Aplicações Desktop



Na década de 90, a maioria dos sistemas era instalada diretamente no computador do usuário.

Máquina Local



- O modelo predominante era o de **duas camadas**: a aplicação cliente **acessava diretamente** o banco de dados.
- Isso limitava muito a escalabilidade e o controle, porque cada máquina **executava a lógica localmente**.

■ Arquitetura Cliente/Servidor Clássica

Com o crescimento das empresas e da necessidade de multiusuários, surge o modelo cliente/servidor.

- A lógica de negócio sai do cliente e passa a ficar em um servidor central.
- O cliente passa a ser responsável principalmente pela interface.



Esse tipo de arquitetura se tornou a base da computação distribuída e permitiu múltiplos usuários acessando o mesmo sistema simultaneamente.

Arquitetura Cliente/Servidor

Exemplos:

- Sistemas bancários com clientes instalados nas agências
- ERPs corporativos com servidor central
- Aplicações Java Swing conectadas a servidores Java EE



■ ————— SOA e Serviços SOAP

- Com o crescimento das **integrações entre empresas**, surge a necessidade de sistemas conversarem entre si de forma padronizada. É nesse cenário que nasce a SOA (Service-Oriented Architecture).
- O **SOAP** foi o **protocolo** mais usado nesse modelo. Ele utiliza XML e contratos formais, **garantindo interoperabilidade** entre plataformas.
- **EXEMPLO:** um ERP que chama um serviço SOAP externo para validar crédito antes de aprovar uma venda.

REST e APIs Web

- Com a popularização da web e do HTTP, surge o estilo REST como uma **alternativa mais simples ao SOAP**.
- REST aproveita o próprio **protocolo HTTP** e usa **formatos leves** como JSON.
- Esse modelo estabelece uma **separação clara entre cliente e servidor** e se torna o padrão dominante das aplicações modernas.
- **Exemplos:** APIs de e-commerce, Backends de aplicativos mobile, Sistemas web corporativos e Plataformas IoT

Antes de avançar!

REST

É um estilo arquitetural usado para construir sistemas distribuídos e, principalmente, APIs web. O nome vem de Representational State Transfer.

Em termos simples, REST define um conjunto de princípios de como sistemas devem se comunicar pela internet, normalmente usando HTTP.

Principais operações:

- GET: consultar dados
- POST: criar
- PUT/PATCH: atualizar
- DELETE: remover

■ --- Microserviços

Microserviços são um estilo de arquitetura de software em que **uma aplicação** é construída como um **conjunto** de **serviços pequenos, independentes e com responsabilidades bem definidas**.

Cada serviço executa uma **função específica de negócio** e pode ser desenvolvido, implantado e escalado de forma independente.

Em vez de um sistema único e grande (monolito), temos vários serviços menores que trabalham juntos para entregar a funcionalidade completa.

Arquitetura Monolítica vs Microserviços

CLIENTE

- Aplicação Única
- Banco de Dados

Características:

- Todas as funcionalidades em um único projeto
- Deploy único
- Escalabilidade total da aplicação
- Forte acoplamento interno

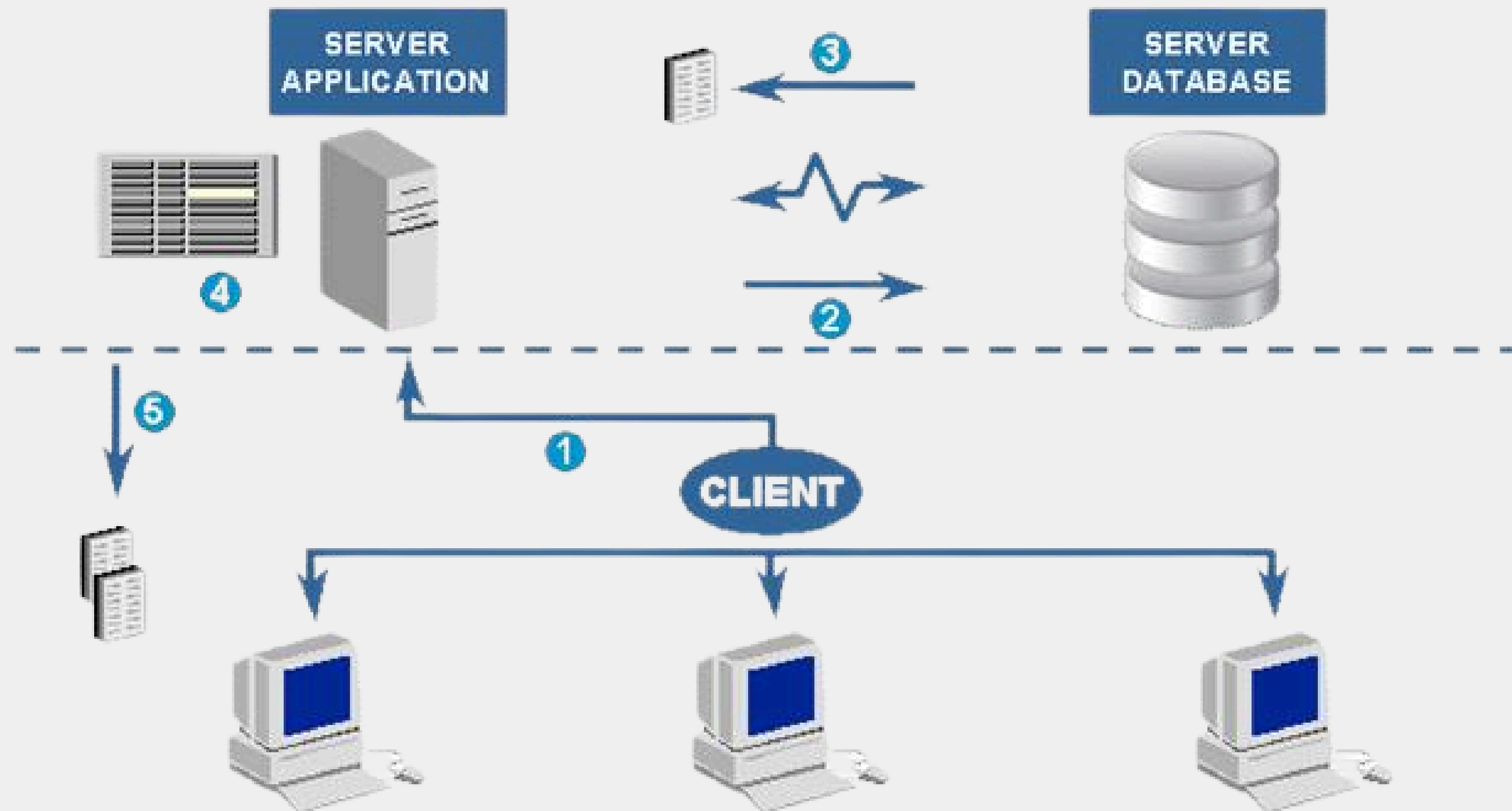
CLIENTE

- API Gateway
- Serviço A
- Serviço B
- Banco(s) independentes

Características:

- Serviços independentes
- Deploy individual
- Escalabilidade específica
- Baixo acoplamento

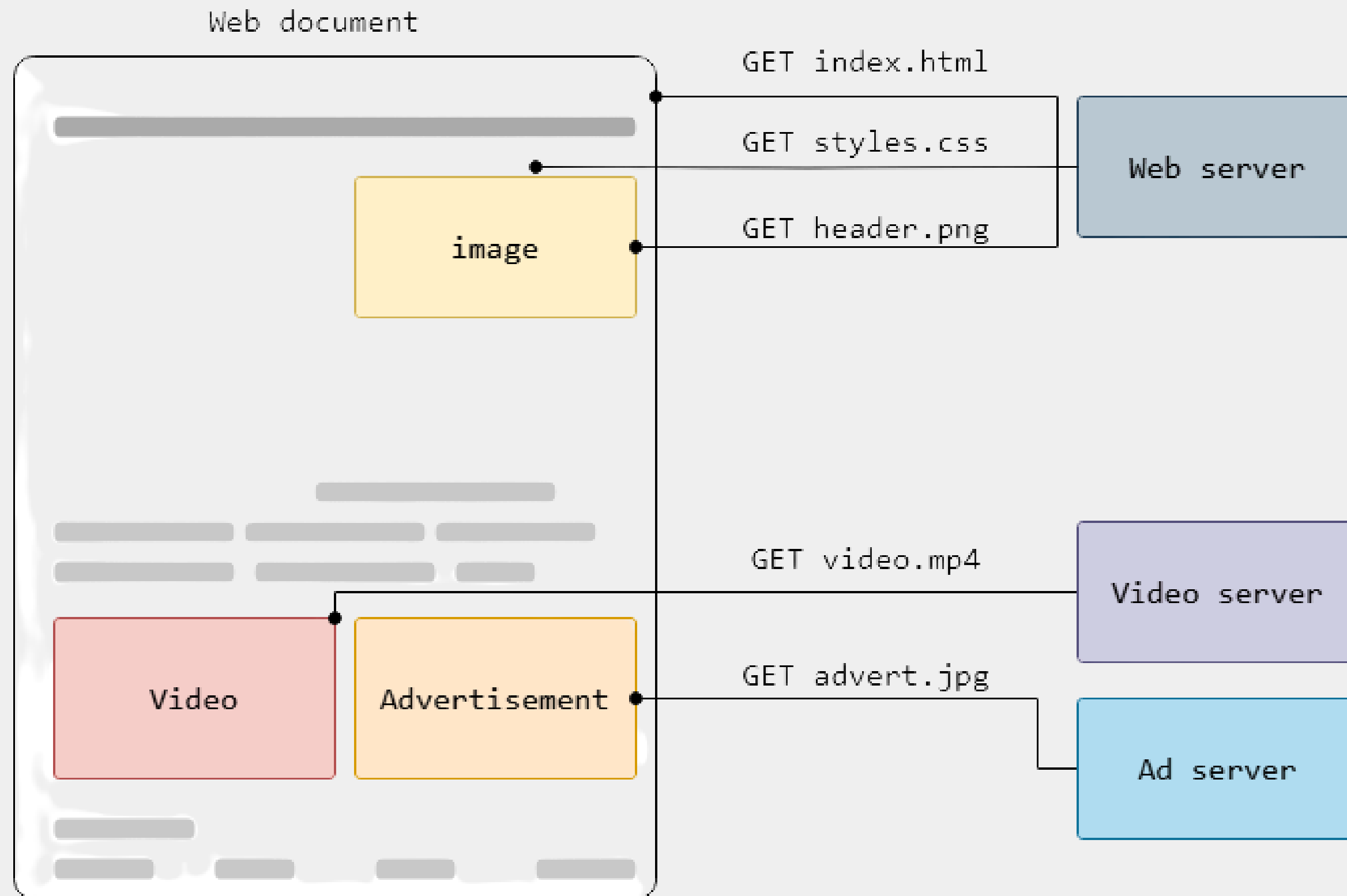
■ Arquitetura Cliente/Servidor



O que é HTTP na prática?

- HTTP é um protocolo que permite a obtenção de recursos, como documentos HTML.
- É a base de qualquer troca de dados na Web e um protocolo cliente-servidor, o que significa que as requisições são iniciadas pelo destinatário, geralmente um navegador da Web.

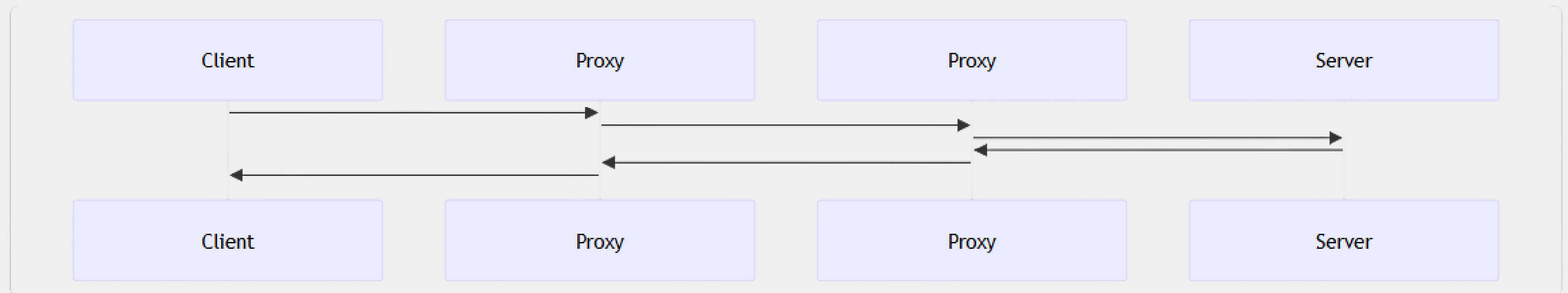
O que é HTTP na prática?



■ Componentes de sistemas baseados em HTTP

- O HTTP é um protocolo cliente-servidor: as requisições são enviadas por uma entidade, o agente-usuário (ou um proxy em nome dele).
- A maior parte do tempo, o agente-usuário é um navegador da Web, mas pode ser qualquer coisa, como por exemplo um robô que varre a Web para preencher e manter um índice de mecanismo de pesquisa e coletar informações.
- Cada requisição individual é enviada para um servidor, que irá lidar com isso e fornecer um resultado, chamado de resposta.

Componentes de sistemas baseados em HTTP



■ Aspectos básicos do HTTP

- Mesmo com mais complexidade introduzida no HTTP/2.0 por encapsular mensagens HTTP em quadros, foi projetado para ser simples e legível às pessoas.
- As mensagens HTTP podem ser lidas e entendidas por qualquer um, provendo uma maior facilidade para desenvolvimento e testes, e reduzir a complexidade para os estudantes.



Fluxo HTTP

Quando o cliente quer comunicar com um servidor, este sendo um servidor final ou um proxy, ele realiza os seguintes passos:

- **Abre uma conexão TCP:** A conexão TCP será usada para enviar uma requisição, ou várias, e receber uma resposta. O cliente pode abrir uma nova conexão, reusar uma conexão existente, ou abrir várias conexões aos servidores.



Fluxo HTTP

Quando o cliente quer comunicar com um servidor, este sendo um servidor final ou um proxy, ele realiza os seguintes passos:

- **Envia uma mensagem HTTP:** mensagens HTTP (antes do HTTP/2.0) são legíveis às pessoas. Com o HTTP/2.0, essas mensagens simples são encapsuladas dentro de quadros (frames), tornando-as impossíveis de ler diretamente, mas o princípio se mantém o mesmo.

Estrutura

- Uma requisição HTTP consiste em uma série de linhas de texto, separadas por `\n`, que delimita uma linha.
- A requisição tem uma linha inicial contendo o método da requisição, o caminho do recurso desejado e a versão do protocolo, seguida por cabeçalhos opcionais e o corpo da requisição.

HTTP

Copy

`GET / HTTP/1.1`

`Host: developer.mozilla.org`

`Accept-Language: fr`

Estrutura

- Uma resposta HTTP também consiste em uma linha inicial contendo a versão do protocolo, o código de status e uma frase descritiva, seguida por cabeçalhos opcionais e o corpo da resposta

HTTP

Copy

HTTP/1.1 200 OK

Date: Sat, 09 Oct 2010 14:28:02 GMT

Server: Apache

Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT

ETag: "51142bc1-7449-479b075b2891b"

Accept-Ranges: bytes

Content-Length: 29769

Content-Type: text/html

<!DOCTYPE html... (here comes the 29769 bytes of the requested web page)

Métodos de Requisição

GET

O método GET é usado para solicitar a recuperação de um recurso específico no servidor. Ele envia os parâmetros na URL, geralmente na forma de cadeias de consulta (query strings). Este é o método mais utilizado na Web, pois é com ele que os browsers fazem a requisição para os servidores.

```
GET /pagina.html?parametro1=valor1&parametro2=valor2 HTTP/1.1  
Host: www.exemplo.com
```

■ Métodos de Requisição

POST

O método POST é usado para enviar dados ao servidor, geralmente para criar um novo recurso ou enviar informações sensíveis, como senhas ou dados de formulários.

```
POST /criar_usuario HTTP/1.1
Host: www.exemplo.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 27

nome=João&email=joao@example.com
```


Métodos de Requisição

PUT

O método PUT é usado para enviar dados ao servidor para criar ou atualizar um recurso específico.

```
PUT /atualizar_usuario/123 HTTP/1.1
Host: www.exemplo.com
Content-Type: application/json
Content-Length: 47

{"nome": "Maria", "email": "maria@example.com"}
```

Métodos de Requisição

DELETE

O método DELETE é usado para solicitar a exclusão de um recurso específico no servidor.

```
DELETE /excluir_usuario/456 HTTP/1.1  
Host: www.exemplo.com
```



Requisições

Method Path Protocol version

GET

/

HTTP/1.1

Host: developer.mozilla.org
Accept-Language: fr

Headers



Respostas

Respostas consistem dos seguintes elementos:

- A versão do protocolo HTTP que elas seguem.
- Um **código de status**, indicando se a requisição foi bem sucedida, ou não, e por quê.
- Uma mensagem de status, uma pequena descrição informal sobre o código de status.
- Cabeçalhos HTTP, como aqueles das requisições.
- Opcionalmente, um corpo com dados do recurso requisitado.



Respostas

